

PÅGÅENDE PROJEKT

Excelakuten

Automatiserad reparation av trasiga Excel-filer.
Ladda upp — få tillbaka en reparerad, validerad fil.

- 7-steps pipeline: inläsning → diagnostik → struktur → tvätt → formelreparation → validering → export
- Ändringslogg i filen: varje cell som rörts, med före/efter och skäl
- Löser vad den kan — rapporterar ärligt vad den inte kan

Python · FastAPI · openpyxl · LibreOffice · Docker-ready

PROBLEMET

Excelfiler som ingen vågar röra

- #REF!-fel och formler som pekar på borttagna blad och gamla nätverksfiler
- Cirkelreferenser som gör att Excel vägrar räkna
- Dolda blad och kolumner som formler i det synliga hämtar siffror ifrån
- Hårdkodade värden inklistrade rakt över levande formler
- Textdatum, tal som text och mellanslag — som ser ut som data men inte är det
- Korrupt XML: filen går inte ens att öppna

Min testfil: 106 000+ formler, 28 000 volatila, tre blad som Excel själv vill "reparera".

Sju steg — varje steg loggat

1

Inläsning

parsar även korrupta filer — CSV, .xls, .ods

2

Diagnostik

felkarta: dolda blad, externa länkar, trasiga kedjor

3

Strukturanalys

hittar tabeller, datatyper, datakällor

4

Datatvätt

textdatum → datum, "1 234,56" → tal, dubbletter bort

5

Formelreparation

#REF!, cirkelreferenser, mönster-ifyllnad

6

Validering

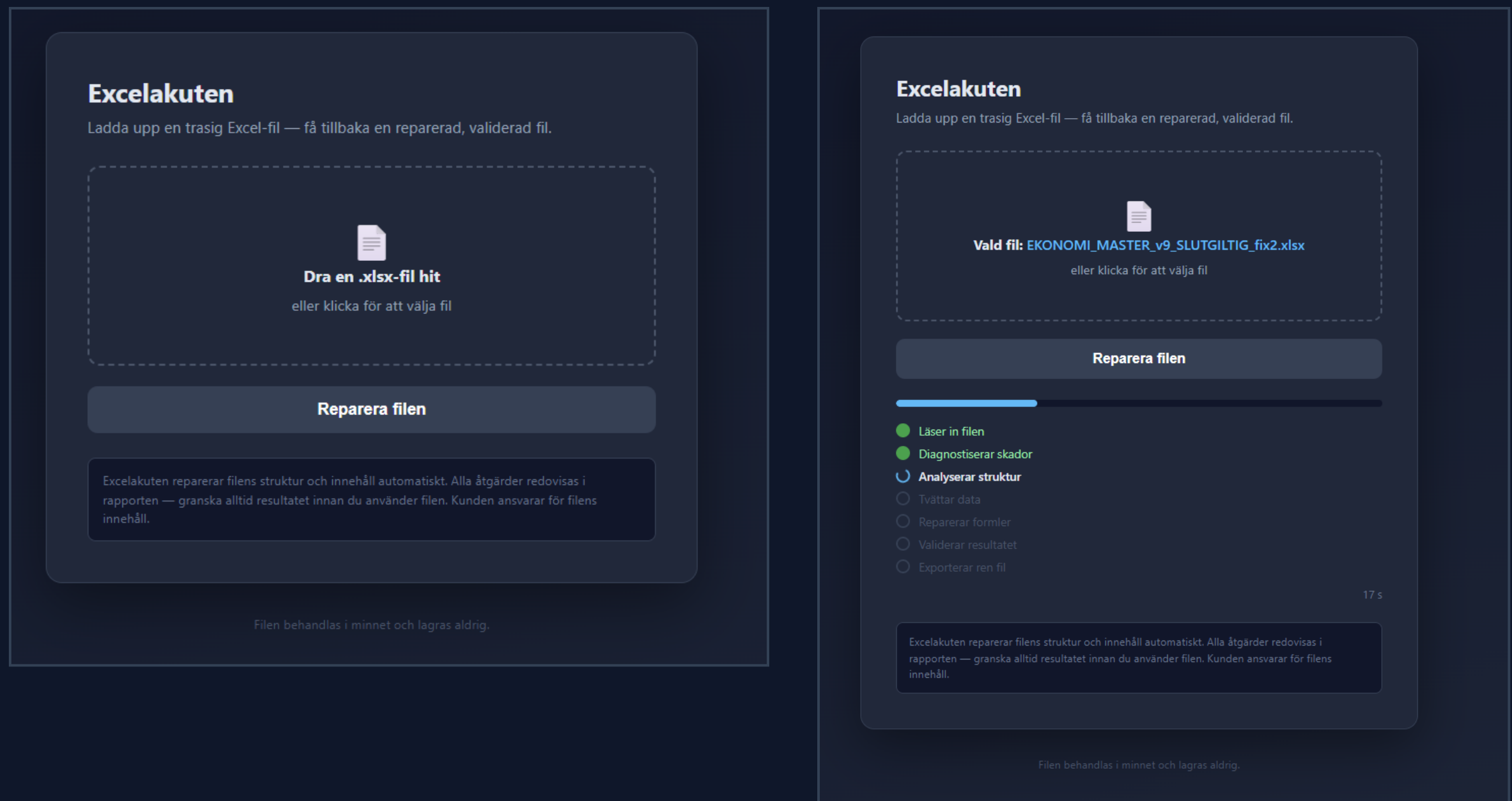
LibreOffice räknar om allt — ny felräkning = stopp

7

Export

ren .xlsx + ändringslogg-flik med varje cell som rörts

Dra in filen — se stegen live



Varje steg rapporteras i realtid via jobb-API:t — ingen magisk spinner.

Filen behandlas i minnet och lagras aldrig.

82 sekunder — allt redovisat

textvärden $\rightarrow$ tal

textdatum $\rightarrow$ datum

cirkelreferenser brutna

formler fick rätt syntax

formler omräknade — 0 nya fel



VARFÖR JAG BYGGDE DET

Berättar vad den INTE kan fixa

83 s

Ren med varningar

Kvarstående problem

X

Kolumner med blandade datatyper kvar (col_B, col_B, col_B, col_B)

X

Formler hämtar data från externa filer (Budget_2013.xls) (651 st)

X

Dolda blad i arbetsboken (2 st)

X

Celler där formler ersatts med fasta värden (2458 st)

Kvarstående problem flaggas

ÄNDRINGSLOGG — Excel Repair Pipeline						
ÄNDRINGSLOGG — Excel Repair Pipeline						
Varje ändrad cell listas med före/efter-värde och skäl. Typ 'gissning' = återskopat/ersatt värde som bör granskas av en människa.						
Blad	Cell	Före	Efter	Ändrad	Typ	
START LÄS MIG	A15	extern konsult (hette nåt med J7), praktikante	extern konsult (hette nåt med J7), praktikan	överflödiga mellanslag borttagna	säker	
START LÄS MIG	A16	chefen (han ändrade ALLT en fredag, återställd	chefen (han ändrade ALLT en fredag, återställd	överflödiga mellanslag borttagna	säker	
Data	K2	=IF(F2="OK",D2,0)	=IF(F2="OK",D2,0)	argumentseparator normaliserad (->.)	säker	
Data	L2	=VLOOKUP(A2:\$A\$1:\$T\$14000;3;FALSE)	=VLOOKUP(A2:\$A\$1:\$T\$14000;3;FALSE)	formel rensad	säker	
Data	N2	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	O2	=IFERROR(D2/B2,"fel")	=IFERROR(D2/B2,"fel")	cell rensad	säker	
Data	Q2	=IFERROR(D2/B2,"fel")	=IFERROR(D2/B2,"fel")	argumentseparator normaliserad (->.)	säker	
Data	B3	2016.04.09	2016-09-04T00:00:00	text konverterad till datum	säker	
Data	N3	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	Q3	=IFERROR(D3/B3,"fel")	=IFERROR(D3/B3,"fel")	argumentseparator normaliserad (->.)	säker	
Data	L4	=VLOOKUP(A4:\$A\$1:\$T\$14000;3;FALSE)	=VLOOKUP(A4:\$A\$1:\$T\$14000;3;FALSE)	formel rensad	säker	
Data	N4	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	Q4	=IFERROR(D4/B4,"fel")	=IFERROR(D4/B4,"fel")	argumentseparator normaliserad (->.)	säker	
Data	B5	24-12-2014	2014-12-24T00:00:00	text konverterad till datum	säker	
Data	D5	88 212.50	88212.5	text konverterad till tal	säker	
Data	I5	4310	4310	text konverterad till tal	säker	
Data	L5	=VLOOKUP(A5:\$A\$1:\$T\$14000;3;FALSE)	=VLOOKUP(A5:\$A\$1:\$T\$14000;3;FALSE)	formel rensad	säker	
Data	N5	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	Q5	=IFERROR(D5/B5,"fel")	=IFERROR(D5/B5,"fel")	argumentseparator normaliserad (->.)	säker	
Data	N6	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	O6	=IFERROR(D6/B6,"fel")	=IFERROR(D6/B6,"fel")	cell rensad	säker	
Data	Q6	=IFERROR(D6/B6,"fel")	=IFERROR(D6/B6,"fel")	argumentseparator normaliserad (->.)	säker	
Data	I7	4310	4310	text konverterad till tal	säker	
Data	N7	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	O7	=IFERROR(D7/B7,"fel")	=IFERROR(D7/B7,"fel")	cell rensad	säker	
Data	Q7	=IFERROR(D7/B7,"fel")	=IFERROR(D7/B7,"fel")	argumentseparator normaliserad (->.)	säker	
Data	I8	4310	4310	text konverterad till tal	säker	
Data	K8	=IF(F8="OK",D8,0)	=IF(F8="OK",D8,0)	argumentseparator normaliserad (->.)	säker	
Data	N8	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	Q8	=IFERROR(D8/B8,"fel")	=IFERROR(D8/B8,"fel")	argumentseparator normaliserad (->.)	säker	
Data	N9	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	Q9	=IFERROR(D9/B9,"fel")	=IFERROR(D9/B9,"fel")	argumentseparator normaliserad (->.)	säker	
Data	I10	4310	4310	text konverterad till tal	säker	
Data	N10	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	=OFFSET(\$D\$1:ROW()-1,0)+RAND()*0	argumentseparator normaliserad (->.)	säker	
Data	Q10	=IFERROR(D10/B10,"fel")	=IFERROR(D10/B10,"fel")	argumentseparator normaliserad (->.)	säker	
Data	B11	2017.11.09	2017-09-11T00:00:00	text konverterad till datum	säker	

Ändringsloggen: varje cell, före/efter

- Inget raderas tyst — rensade celler loggas med cellreferens
- Fliken "Ändringslogg" i utfilen: före/efter + skäl, per cell
- Integritetscheck diffar varje cell mot originalet — oförklarade ändringar flaggas

Byggd som Tier-1 av en större vision:

förstå filens datakällor — automatisera hela pipelinen bakom den.